Отчет по лабораторной работе №1:

Полнодоступная двухсервисная модель Эрланга с одинаковыми интенсивностями обслуживания

Дисциплина: Основы проектирования сетей и систем телекоммуникаций

Выполнил: Кхари Жекка Кализая Арсе, НПИбд-01-23

Содержание

1 цель **3**

2 Численный анализ **4**
2.1 Пошаговая работа алгоритма 8
2.2 пример работы 10

3 исходный код **13**

1 цель

Смоделировать перегрузку сети, обслуживающей два сервиса без очереди.

2 Численный анализ

Для моделирования этого сценария сначала были подготовлены исходные данные:

- C: пиковая пропускная способность соты
- λ_1 и λ_2: интенсивность поступления запросов на предоставление услуги 1, 2-го типа [запросов/ед.вр.];
- U: среднее время обслуживания запроса на предоставление услуги 1, 2-го тип [запросов/ед.вр.];

Остальные данные будут вычислены и занесены в таблицу, представленную на Рисунок 2.1.

simulation_N	L1	L2	C	U	E	N	B1	B2	L
1	0.2	0.8	5	5.0	0.000002	0.200000	4.366564e-07	0.000002	1.0
2	0.4	1.6	5	5.0	0.000057	0.399977	1.144018e-05	0.000046	2.0
3	0.6	2.4	5	5.0	0.000356	0.599787	7.112875e-05	0.000285	3.0
4	0.8	3.2	5	5.0	0.001227	0.799018	2.454388e-04	0.000982	4.0
5	1.0	4.0	5	5.0	0.003067	0.996933	6.134969e-04	0.002454	5.0
6	1.2	4.8	5	5.0	0.006255	1.192494	1.250989e-03	0.005004	6.0
7	1.4	5.6	5	5.0	0.011088	1.384477	2.217528e-03	0.008870	7.0
8	1.6	6.4	5	5.0	0.017749	1.571601	3.549839e-03	0.014199	8.0
9	1.8	7.2	5	5.0	0.026302	1.752657	5.260317e-03	0.021041	9.0
10	2.0	8.0	5	5.0	0.036697	1.926606	7.339450e-03	0.029358	10.0
11	2.2	8.8	5	5.0	0.048802	2.092635	9.760442e-03	0.039042	11.0
12	2.4	9.6	5	5.0	0.062423	2.250185	1.248457e-02	0.049938	12.0

Рисунок 2.1: таблица данных

Алгоритм состоит из двух основных функций: netSimulator и functionPn.

- netSimulator Рисунок 2.2: предназначена для работы с переменной data, которая содержит минимально необходимую информацию для вычисления:
  - E (Вероятность блокировки по времени)
  - P_{n..c} (Распределение вероятностей состояний системы)
  - N (Среднее число обслуживаемых в системе запросов)
  - B_{1,2} Вероятность блокировки по вызовам для услуг

```python
def netSimulator(data):

    C = data["C"]
    L1 = data["L1"]
    L2 = data["L2"]
    U = data["U"]

    p1 = L1/U
    p2 = L2/U
    E = 0
    aceptedQuerysAverage = 0
    Pdistribution = []
    for i in range(C+1):

        Pn = functionPn(p1, p2, C, i)
        Pdistribution.append(Pn)S
        aceptedQuerysAverage += i * Pn

        if i == C:
            E = Pn
            B = (((L1/(L1+L2)) * E) , ((L2/(L1 + L2)) *E))


    return E, Pdistribution, aceptedQuerysAverage, B
```

Рисунок 2.2: функция netSimulator

- functionPn Рисунок 2.3: разработана как вспомогательная функция для netSimulator. Она вычисляет сумму в формуле $p_n = \left(\sum_{i=0}^{C} \frac{(\rho_1+\rho_2)^i}{i!}\right)^{-1} \cdot \frac{(\rho_1+\rho_2)^n}{n!}$ и принимает значения p1 и p2 (интенсивность предложеннойнагрузки предоставление услуги 1, 2-го типа), C и N.

```python
def functionPn(p1, p2, C, n):
    summa = 0
    for i in range(C+1):
        summa += (p1 + p2) ** i / factorial(i)

    return (p1 + p2) ** n / factorial(n) / summa
```

Рисунок 2.3: функция functionPn

- функция drawPlot Рисунок 2.4: предназначена для построения графика зависимости вероятности блокировки от интенсивности

поступления запросов на обслуживание и графика зависимости среднего числа обслуживаемых запросов от интенсивности поступления запросов на предоставление услуги.

```python
def drawPlots( data:pd.DataFrame, column:str, type:int):


    titles = {
    1:"график зависимости вероятности блокировки\n от интенсивности поступления запросов на обслуживание.",
    2:"график зависимости среднего числа обслуживаемых\n запросов от интенсивности поступления запросов на предоставление услуги."
    }

    labelsY = {
       1: "вероятности блокировки",
        2:"среднего числа обслуживаемых запросов"
    }
    labelsX = {
        1:"интенсивности поступления запросов на обслуживание.",
        2:"интенсивности поступления запросов на предоставление услуги."
    }


    plt.figure(figsize=(10, 5))
    plt.bar(data["L"] , data[ column] )
    plt.title( titles[type] )
    plt.xlabel(labelsX[type] )
    plt.ylabel(labelsY[type])
    #plt.text(5, 0.1, f"E = {E}", fontsize=10, verticalalignment='top', bbox=estilo_recuadro)
    #plt.text(5, 0.05, f"Average Accepted Queries = {aceptedQuerysAverage}", fontsize=10, verticalalignment='top', bbox=estilo_recuadro)
    plt.show()
```

Рисунок 2.4: функция drawPlot

- главная функция: выполняет операции ввода с клавиатуры, чтобы пользователь мог вручную ввести минимально необходимые значения для запуска моделирования, вызвать описанные выше функции, сохранить данные и вывести их в консоль.

```
print("программа моделирования насыщения сети")

C = 0
while(C <= 0):
    C = int( input("введите значение C\n"))
    if (C <=0):
        print("C должно быт больше 0")

L1 = 2
while(L1>1 or L1<=0):
    L1 = float(input("введите соотношение между L1 и L2\nнапримет 0.2\n"))
    if L1 > 1:
        print("неправильное значение")

U = 0
while(U <= 0):
    U = float(input("введите здачение U\n"))
    if(U == 0):
        print("U должно быт больше 0")

numberOfSimulation = int(input("введите максимальный L\n"))


data = {
    "simulation_N": [i + 1 for i in range(numberOfSimulation)],
    "L1": [ (i+1)*L1 for i in range(numberOfSimulation) ],
    "L2": [ (i+1)*(1-L1) for i in range(numberOfSimulation)],
    "C": np.full(numberOfSimulation,C),
    "U": np.full(numberOfSimulation, U ),
    "E": np.zeros(numberOfSimulation),
    "N": np.zeros(numberOfSimulation),
    "B1":np.zeros(numberOfSimulation),
    "B2":np.zeros(numberOfSimulation)

}

data["L"] = np.array(data["L1"]) + np.array(data["L2"])
Pdistributions = []

for i in range(numberOfSimulation):
```

```
    simulation_data = {
        "C": data["C"][i],
        "L1": data["L1"][i],
        "L2": data["L2"][i],
        "U": data["U"][i]
    }

    E, Pdistribution, aceptedQuerysAverage, B = netSimulator(simulation_da

    data["E"][i] = E
    data["N"][i] = aceptedQuerysAverage
    data["B1"][i] = B[0]
    data["B2"][i] = B[1]
    Pdistributions.append (Pdistribution)



dt = pd.DataFrame(data)
Pdistributions = pd.DataFrame(Pdistributions)



print("data:\n",dt)
print("\nPn:",Pdistributions)

drawPlots(dt, "E",type=1)
drawPlots(dt,"N", type=2)
```

2.1 Пошаговая работа алгоритма

Как указано в задании к лабораторной работе № 1, будут использоваться следующие основные формулы:

$\rho_i = \frac{\lambda_i}{\mu}$

$$p_n = \left(\sum_{i=0}^{C} \frac{(\rho_1+\rho_2)^i}{i!} \right)^{-1} \cdot \frac{(\rho_1+\rho_2)^n}{n!}$$

$$P_c = E$$

$$B_n = \frac{\lambda_n}{\lambda_1+\lambda_2}$$

$$\bar{N} = \sum_{n \in X} n p_n$$

1. Вычисляются ρ_1 и ρ_2. Поскольку они представляют нагрузку, их вычисление упрощается до $\frac{\lambda_n}{\mu}$.

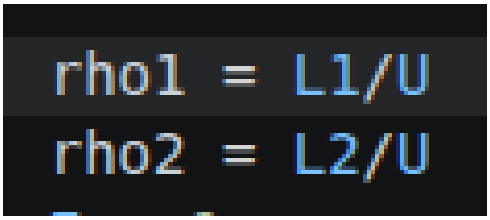


Рисунок 2.5: вычисление ρ_1 и ρ_2

2. Инициализируются переменные для хранения данных: Вероятность блокировки по времени (E), Среднее число обслуживаемых в системе запросов и Распределение вероятностей состояний системы (Pdistribution), а также aceptedQuerysAverage.

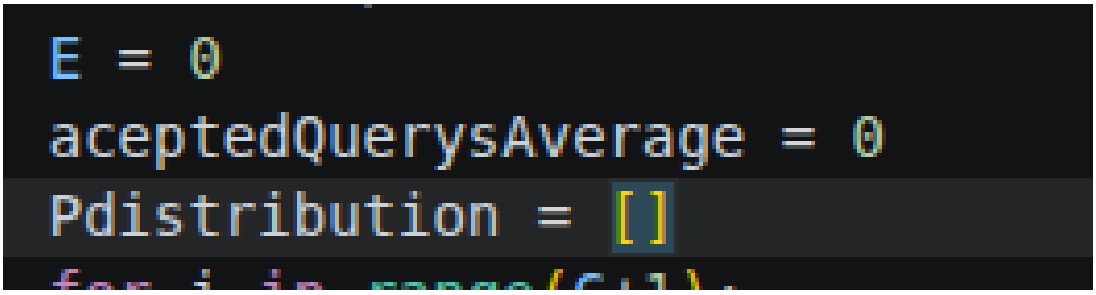


Рисунок 2.6: инициализация переменных

3. Выполняется цикл for для вычисления всех P_n. Для этого вызывается функция functionPn() с соответствующими аргументами, после чего она вычисляет формулу, указанную в ее описании. На каждом шаге значение P_i сохраняется в Pdistribution, а по формуле $\bar{N}$ вычисляется соответствующее Среднее число обслуживаемых в системе запросов. Когда i становится равным C, то есть $P_n = P_c$, это значение сохраняется в переменной E. Затем в кортеже B вычисляются и сохраняются значения B_1 и B_2. В итоге эти переменные возвращаются, записываются в таблицу данных, после чего начинается следующая симуляция с большим значением λ.

```python
for i in range(C+1):

    Pn = functionPn(rho1, rho2, C, i)
    Pdistribution.append(Pn)
    aceptedQuerysAverage += i * Pn

    if i == C:
        E = Pn
        B = (((L1/(L1+L2)) * E) , ((L2/(L1 + L2)) *E))


return E, Pdistribution, aceptedQuerysAverage, B
```

Рисунок 2.7: цикл for

2.2 пример работы

Выполним программу с начальными значениями:

C = 5, U = 0.5, L1/L2 = 0.2 и Lmax = 10

Для них были получены следующие результаты:

```
программа моделирования насыщения сети
введите значение C
5
введите соотношение между L1 и L2
напримет 0.2
0.2
введите здачение U
0.5
введите максимальный L
10
data:
    simulation_N   L1   L2  C    U         E         N        B1        B2     L
0              1  0.2  0.8  5  0.5  0.036697  1.926606  0.007339  0.029358   1.0
1              2  0.4  1.6  5  0.5  0.199067  3.203733  0.039813  0.159253   2.0
2              3  0.6  2.4  5  0.5  0.360400  3.837597  0.072080  0.288320   3.0
3              4  0.8  3.2  5  0.5  0.479008  4.167934  0.095802  0.383207   4.0
4              5  1.0  4.0  5  0.5  0.563952  4.360478  0.112790  0.451162   5.0
5              6  1.2  4.8  5  0.5  0.626352  4.483779  0.125270  0.501081   6.0
6              7  1.4  5.6  5  0.5  0.673675  4.568557  0.134735  0.538940   7.0
7              8  1.6  6.4  5  0.5  0.710621  4.630056  0.142124  0.568497   8.0
8              9  1.8  7.2  5  0.5  0.740192  4.676544  0.148038  0.592154   9.0
9             10  2.0  8.0  5  0.5  0.764358  4.712840  0.152872  0.611486  10.0

Pn:           0         1         2         3         4         5
0  0.137615  0.275229  0.275229  0.183486  0.091743  0.036697
1  0.023328  0.093313  0.186625  0.248834  0.248834  0.199067
2  0.005562  0.033370  0.100111  0.200222  0.300334  0.360400
3  0.001754  0.014033  0.056134  0.149690  0.299380  0.479008
4  0.000677  0.006767  0.033837  0.112790  0.281976  0.563952
5  0.000302  0.003625  0.021748  0.086993  0.260980  0.626352
6  0.000150  0.002104  0.014730  0.068742  0.240598  0.673675
7  0.000081  0.001301  0.010409  0.055517  0.222069  0.710621
8  0.000047  0.000846  0.007615  0.045691  0.205609  0.740192
9  0.000029  0.000573  0.005733  0.038218  0.191089  0.764358
```

Рисунок 2.8: итоговая таблица данных

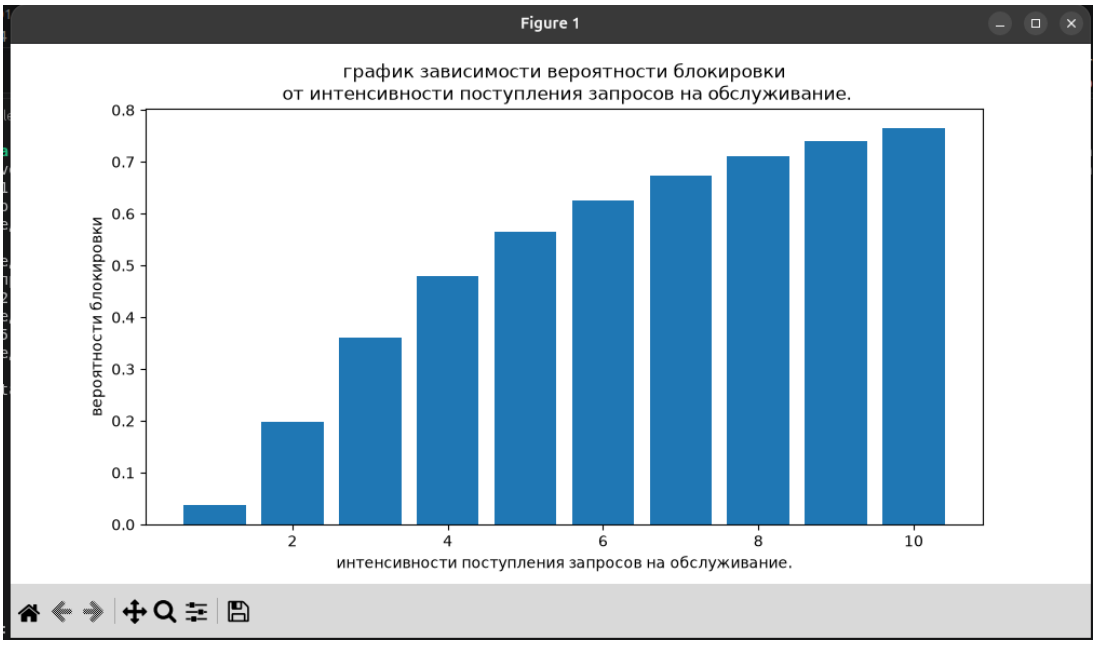


Рисунок 2.9: график зависимости вероятности блокировки от интенсивности поступления запросов на обслуживание.

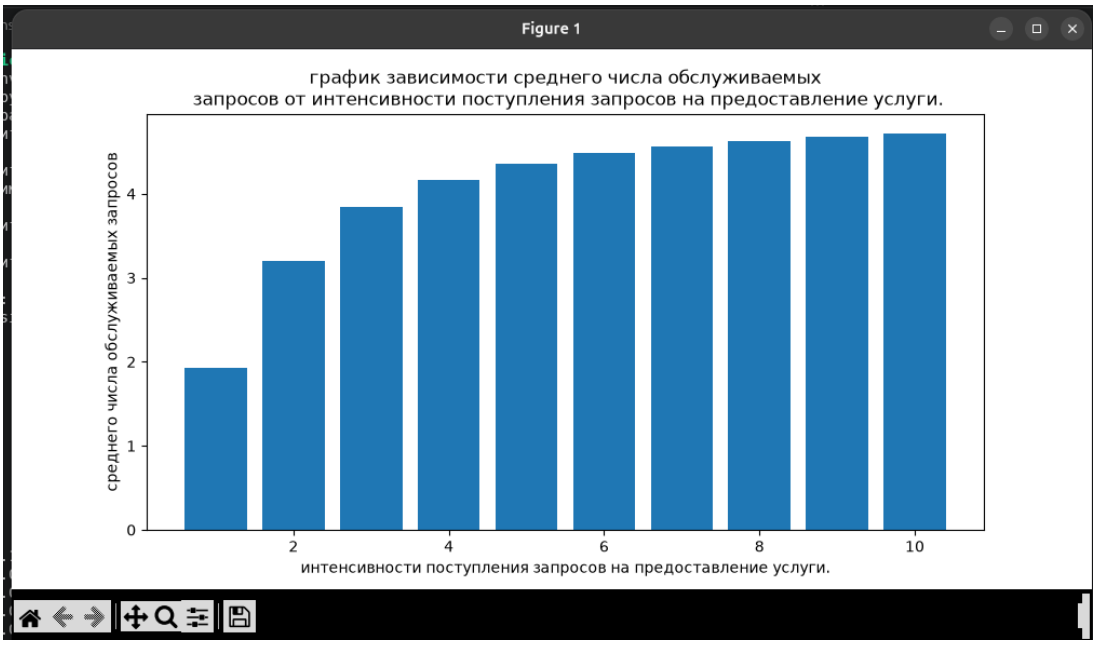


Рисунок 2.10: график зависимости среднего числа обслуживаемых запросов от интенсивности поступления запросов на предоставление услуги.

3 исходный код

```
from math import factorial
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import argparse

def functionPn(rho1, rho2, C, n):
    summa = 0
    for i in range(C+1):
        summa += (rho1 + rho2) ** i / factorial(i)

    return (rho1 + rho2) ** n / factorial(n) / summa

def netSimulator(data):

    C = data["C"]
    L1 = data["L1"]
    L2 = data["L2"]
    U = data["U"]

    rho1 = L1/U
    rho2 = L2/U
    E = 0
    aceptedQuerysAverage = 0
    Pdistribution = []
    for i in range(C+1):

        Pn = functionPn(rho1, rho2, C, i)
        Pdistribution.append(Pn)
        aceptedQuerysAverage += i * Pn

        if i == C:
            E = Pn
            B = (((L1/(L1+L2)) * E) , ((L2/(L1 + L2)) *E))


    return E, Pdistribution, aceptedQuerysAverage, B

def drawPlots( data:pd.DataFrame, column:str, type:int):
```

```
    titles = {
  1:"график зависимости вероятности блокировки\n от интенсивности поступлени
  2:"график зависимости среднего числа обслуживаемых\n запросов от интенсивн
    }

    labelsY = {
      1: "вероятности блокировки",
        2:"среднего числа обслуживаемых запросов"
    }
    labelsX = {
       1:"интенсивности поступления запросов на обслуживание.",
    2:"интенсивности поступления запросов на предоставление услуги."
    }


    plt.figure(figsize=(10, 5))
    plt.bar(data["L"] , data[ column] )
    plt.title( titles[type] )
    plt.xlabel(labelsX[type] )
    plt.ylabel(labelsY[type])
  #plt.text(5, 0.1, f"E = {E}", fontsize=10, verticalalignment='top', bbox=e
  #plt.text(5, 0.05, f"Average Accepted Queries = {aceptedQuerysAverage}", f
    plt.show()




#E, Pdistribution, aceptedQuerysAverage = netSimulator(10, 5, 3, 20)

 print("программа моделирования насыщения сети")

 C = 0
 while(C <= 0):
     C = int( input("введите значение C\n"))
     if (C <=0):
         print("C должно быт больше 0")

 L1 = 2
 while(L1>1 or L1<=0):
  L1 = float(input("введите соотношение между L1 и L2\nнапримет 0.2\n"))
     if L1 > 1:
         print("неправильное значение")

 U = 0
```

```
    while(U <= 0):
        U = float(input("введите здачение U\n"))
        if(U == 0):
            print("U должно быт больше 0")

    numberOfSimulation = int(input("введите максимальный L\n"))


    data = {
        "simulation_N": [i + 1 for i in range(numberOfSimulation)],
        "L1": [ (i+1)*L1 for i in range(numberOfSimulation) ],
        "L2": [ (i+1)*(1-L1) for i in range(numberOfSimulation)],
        "C": np.full(numberOfSimulation,C),
        "U": np.full(numberOfSimulation, U ),
        "E": np.zeros(numberOfSimulation),
        "N": np.zeros(numberOfSimulation),
        "B1":np.zeros(numberOfSimulation),
        "B2":np.zeros(numberOfSimulation)

    }

    data["L"] = np.array(data["L1"]) + np.array(data["L2"])
    Pdistributions = []

    for i in range(numberOfSimulation):
        simulation_data = {
            "C": data["C"][i],
            "L1": data["L1"][i],
            "L2": data["L2"][i],
            "U": data["U"][i]
        }

     E, Pdistribution, aceptedQuerysAverage, B = netSimulator(simulation_data)

        data["E"][i] = E
        data["N"][i] = aceptedQuerysAverage
        data["B1"][i] = B[0]
        data["B2"][i] = B[1]
        Pdistributions.append (Pdistribution)



    dt = pd.DataFrame(data)
    Pdistributions = pd.DataFrame(Pdistributions)



    print("data:\n",dt)
```

```
    print("\nPn:",Pdistributions)

    drawPlots(dt, "E",type=1)
    drawPlots(dt,"N", type=2)
```